

ArchAI Novelty Accuracy and Viva Guide

Code backed explanation of deterministic requirement engineering, accuracy safeguards, dynamic generation, bug corrections, and viva answers

Purpose. This guide explains what is original in ArchAI beyond calling Ollama or exposing APIs. It maps every defensible claim to code, explains how functional and non-functional requirements are separated, states what is dynamic and what is deterministic, records the bugs corrected, and prepares the presenter for technical questioning.

Verified state. 494 backend tests passed with Ollama disabled; 39 frontend tests passed; TypeScript compilation and the production build succeeded. The 44/44 classification result is a repository regression benchmark, not a claim of universal real-world accuracy.

How to use this guide

- Read Parts II and III to understand the algorithms and novelty.
- Use the bug ledger to explain the engineering process and evidence.
- Memorise the short answers and numbers in Part IV before the viva.
- Demonstrate an unseen domain with Ollama disabled to prove dynamic behavior.
- Never claim 100 percent general accuracy; state the benchmark scope and limitations.

Part II - Novelty, Accuracy, and How the Code Actually Works

This part is written to be defended out loud. Every claim names the file and function it comes from, and every number is produced by a test in the repo that anyone can re-run. Where the system is weak, it says so - an examiner will find the weak cases anyway, and knowing them is stronger than claiming they do not exist.

1. The one-sentence answer to "what is novel here?"

Anyone can call an LLM and ask it for an architecture. The novelty here is that the language model is not allowed to decide anything, and every artifact is traceable to a sentence the user actually wrote.

Concretely, four things:

1. **A deterministic requirement extractor that works with the LLM switched off entirely.** The whole backend test suite - 494 tests - runs with `ARCHAI_OLLAMA_ENABLED=false`. If the model were doing the work, the suite could not exist.
2. **Source grounding as a hard gate.** Anything the model returns that is not traceable to the brief is deleted before it is stored, by rule, not by prompt instruction (`_strip_unsupported_claims`).
3. **A requirement-to-artifact causal graph**, so every component, table, endpoint and diagram element can be traced back to the requirement that caused it, and the blast radius of a change is computed by graph traversal rather than by asking a model.
4. **Diagrams that obey UML, and refuse to state what the model does not know.** Where the requirement model has no answer, the diagram says "Actor not confirmed" instead of inventing a plausible actor.

The difference is testable. A system that asks a model for an architecture cannot be unit-tested, because the same brief gives a different answer each run. This one is deterministic, so it can be - and that is the point.

2. Static or dynamic? (the question that will definitely be asked)

It is dynamic. There are four *domain labels* built in, but there is no built-in *content* for any of them.

This distinction is the whole design, so it is worth being precise.

`RequirementAnalyzer.analyze()` (`app/services/requirement_analyzer.py:290`) picks one of three paths, and records which one it used in `RequirementModel.analysis_source` - visible in the UI as the badge on the Requirements page:

<code>`analysis_source`</code>	When	What it produces
predefined-blueprint	<code>_pick_blueprint()</code> recognises the domain (EV charging, online pharmacy, e-commerce, learning)	The domain label only . Requirements still come from the brief.
deterministic-extraction	No blueprint matched, but the brief has extractable structure	Requirements, actors, entities, workflows - all derived from the brief by rule
conservative-fallback	The brief has no extractable structure at all	The brief kept verbatim, domain marked unknown, clarification questions raised
ollama-pretrained	Ollama is reachable <i>and</i> its output survives validation and grounding	The same shape, semantically interpreted, then filtered

The key point about blueprints

A blueprint is **a classifier, not a content source**. The code says so directly (`requirement_analyzer.py`, `_analyze_known`):

```
# A blueprint is a fast domain classifier, never a source of product
# requirements. Earlier versions copied every blueprint feature,
# actor, entity, and compliance item into the project, so mentioning
# an online pharmacy could invent checkout, payments, inventory, and
# admin workflows. Reuse the grounded deterministic extractor and
# only apply the confidently matched domain label.
grounded = self._analyze_deterministic(...)
```

So a brief that says "online pharmacy" does **not** inherit a pharmacy feature list. It gets the label `Online Pharmacy` and then its own requirements are extracted from its own words. The same used to be true of the diagrams: 329 lines of hand-written EV-charging diagram content sat behind `if requirements.domain == "EV Charging Booking Platform"` in `diagram_generator.py`. That code is deleted. Nothing in the diagram layer now branches on a domain name.

If asked to prove it: `tests/test_diagram_notation.py` generates five projects that share no vocabulary (EV charging, pharmacy, LMS, freight logistics, IT service desk) and asserts that each project's diagrams contain its own domain words and **none of the other four's**. A hardcoded template would fail that test instantly.

How to demonstrate it live

Give it a domain the system has never seen - "a grant management system where reviewers score applications, committees approve awards, and finance teams disburse funds" - with Ollama switched off. It returns four requirements, three actors (Reviewer, Committee, Finance Team) and seven diagrams. No blueprint matched; nothing was templated.

3. How functional and non-functional requirements are separated

File: `backend/app/services/requirement_analyzer.py` **Function:** `_reclassify_requirements()` (~line 1273)

Tests: `backend/tests/test_requirement_classification.py`

This is the part most likely to be probed, so here is the full mechanism.

The problem it solves

Extraction - whether by rule or by model - produces a *label* for each statement. That label is often wrong, because the giveaway words are misleading:

- "Support security, residency, and auditability." starts with a verb, so it looks functional. It is three quality attributes.
- "The service must meet a 99.95% availability target." has a modal, so it looks like a constraint. It is a quality target.
- "Display live bay availability for each depot." contains the word "availability", so it looks non-functional. It is a feature.

So the classifier **re-decides every statement from its text** and ignores the label it arrived with. The docstring states this: *"Re-evaluate model categories instead of trusting the output label."*

The three signals

It computes three independent, cheap signals per statement:

``_quality_hits(value) -> int`` - how much quality evidence is present:

- counts matches against `QUALITY_REQUIREMENT_MARKERS` (41 terms: latency, availability, encrypt, audit, resilience, observability, throughput, ...)
- +1 for a measured figure with a unit: `\d+\s*(%|ms|milliseconds|seconds)`
- +1 for a load figure: `200,000 concurrent users, 5 million events per day`

A **count**, not a boolean - because one quality word is weak evidence and two is strong. That threshold is used below.

``_solution_constraint(value) -> bool`` - an **AND of two independent signals**:

- a hard modal: `must, must not, cannot, never, only, shall, required`
- **and** a solution-space marker from `SOLUTION_CONSTRAINT_MARKERS` (22 terms: `must use, must deploy, in-country, compliance, regulation, strongly consistent, ...`)

The AND is what makes it safe. "Operators must approve refunds." has the modal but names no technology, region or regulation, so it is not a constraint - it stays a feature.

``_domain_actions(value) -> int`` - how many domain verbs appear (`approve, book, dispatch, settle, submit, transfer, capture, ...`), matched with `\b<verb>w*\b` so "books", "booking" and "booked" all count.

The disambiguator that makes "availability" work

`_resource_availability_capability(value) -> bool` exists because "availability" means two different things:

```
# Not a capability - these are system-uptime phrasings
if any(marker in lower for marker in (
    "availability target", "uptime", "service availability",
    "system availability", "platform availability",
)):
    return False
# A capability - someone is looking at a resource's availability
return bool(re.search(
    r"\b(?:check|display|expose|find|search|show|track|view)w*\b"
    r".{0,80}\b(?:availability|available)\b",
    lower,
))
```

So "Display live bay availability" is functional and "99.95% availability target" is not. This is exactly the kind of case a single keyword list gets wrong, and it is why the classifier is a set of interacting rules rather than a word list.

The decision order

The order matters and is deliberate. For a statement currently labelled functional:

```

if _solution_constraint(item):
    -> constraint
elif quality_hits and not _resource_availability_capability(item) and (
    numeric_quality          # it states a measured figure
    or quality_hits >= 2      # two or more quality attributes
    or domain_actions == 0    # nothing anyone actually does
    or starts_with_support_provide_maintain
):
    -> non-functional
else:
    -> functional          # the default: behaviour

```

Read out loud: *"A hard external limit is a constraint. Otherwise, quality evidence makes it non-functional only if it is measured, or compound, or has no action in it at all, or is phrased as a blanket 'support X'. Everything else is behaviour."*

Statements arriving as non-functional and as constraints go through the mirror image of the same rules, so a statement lands in the same bucket regardless of which category it arrived in - a property worth stating, because it means the classifier is a function of the text, not of the extractor's mood.

Finally, a de-duplication pass with a documented precedence: **a statement that appears in two categories stays functional**.

```

# A capability statement duplicated across categories stays functional:
# drop the NFR copy, never the validated functional requirement.

```

Measured accuracy - with the honest caveats

tests/test_requirement_classification.py holds a **44-statement labelled benchmark**: 21 functional, 15 non-functional, 8 constraints, each labelled by what a requirements engineer would call it, independent of what the code does. Every statement is fed in **labelled functional** - the hardest direction, because the classifier must pull qualities and constraints out unaided.

Class	Score
Functional	21 / 21
Non-functional	15 / 15
Constraints	8 / 8
Overall	44 / 44 (100%)

Say this out loud with the caveat, because it is the honest framing: 100% is on *this* benchmark, which is 44 statements written by us. It is a regression guard, not a claim of general accuracy. What it does establish is that the classifier is **deterministic and measurable** - the same statement always lands in the same bucket, so the number means something and a regression fails a test. A model-based classifier cannot give that guarantee at all.

Before the bug fixes described in Part III, the first 43 of these statements scored **36/43, with constraint recall at 2/8**. The benchmark is what found that.

4. What "deterministic" means here, and why it is the right call

"Deterministic" means: same input -> same output, every time, computed by code you can read and step through. No sampling, no temperature, no network.

What is deterministic, and why

Concern	Why it must not be a model's judgement
FR / NFR / constraint classification	Must be stable and explainable to a stakeholder
Dependency traversal, blast radius	A wrong answer is silently wrong; graph reachability is exact
Architecture scoring (12 metrics)	A recommendation must be reproducible and auditable
Cost and complexity estimates	Arithmetic, not opinion
CRUD, persistence, undo/redo, permissions	Correctness is not negotiable
Validation	The thing that catches the model's mistakes cannot itself be the model
Diagram construction	Notation is a specification, not a style

What the model is for

Semantic understanding of unusual prose, ambiguity resolution, explanation wording, and prototype structure suggestions. Everything it returns is **validated and grounded before it is stored** (Section 5).

The strongest single piece of evidence

The entire 494-test backend suite runs with Ollama disabled (`tests/conftest.py` sets `ARCHAI_OLLAMA_ENABLED=False`), and the application is fully usable in that state - requirements, architectures, scoring, database, APIs, diagrams, prototype, causal graph, reports. If the intelligence lived in the prompt, none of that would work.

Line counts, as a rough measure of where the work is:

File	Lines	What it decides deterministically
<code>requirement_analyzer.py</code>	3,256	Extraction, FR/NFR/constraint classification, actor validation, grounding
<code>assistant_intel.py</code>	2,126	Intent resolution and grounded lookups before any model call
<code>domain_inference.py</code>	1,847	Sentence and clause decomposition, actor/entity/capability extraction

File	Lines	What it decides deterministically
diagram_generator.py	1,819	Seven diagram types in correct UML notation
database_generator.py	1,430	Schema, keys, relationship direction, conflict resolution
causal_graph.py	1,055	Requirement -> artifact traceability and blast radius
comparison_engine.py	725	12-metric weighted scoring
prototype_generator.py	642	Screen composition from entities, actors and workflows

For comparison, `ai/client.py` - the entire Ollama integration - is one file whose failure path is a single `logger.info("Skipping Ollama refinement...")`. The system is designed so that losing the model degrades *richness*, never *correctness*.

5. Source grounding: how invention is prevented

File: `requirement_analyzer.py`, `_strip_unsupported_claims()` (~line 2635)

Any text the model produces passes five independent checks. **If any one fires, the text is discarded** - not softened, not flagged, discarded.

```
def _strip_unsupported_claims(self, value: str, source_text: str) -> str:
    if self._unsupported_numbers(value, source_text): return ""
    if self._unsupported_technologies(value, source_text): return ""
    if self._unsupported_characteristics(value, source_text): return ""
    if self._unsupported_actions(value, source_text): return ""
    if self._unsupported_unknown_claims(value, source_text): return ""
    return " ".join(value.split()).strip()
```

`_unsupported_numbers` is the clearest to explain in a viva - it is four lines and impossible to argue with:

```
output_numbers = set(re.findall(r"\b\d+(?:[.]\d+)?%\b", value.lower()))
source_numbers = set(re.findall(r"\b\d+(?:[.]\d+)?%\b", source_text.lower()))
return sorted(output_numbers - source_numbers)
```

Every number in the output must appear in the input. A model that helpfully suggests "99.9% uptime" for a brief that never mentioned a target has that requirement deleted. This is why the project can promise it does not invent availability targets, traffic levels, budgets or regions - it is set difference, not a prompt asking nicely.

Actor grounding, and the trap it avoids

`_tokens_fully_grounding()` requires **every** significant token of a name to appear in the brief, not just the head noun:

```
"""Ollama often invents qualified names ("Sales Manager", "Financial Record")
where only the head noun exists in the brief. Head-only checks let those
through; full-token grounding rejects an invented qualifier while keeping
genuine brief participants (pluralization-insensitive)."""
```

A brief mentioning "managers" does not license an actor called "Sales Manager".

6. Where the novelty lives, file by file

Points 1-5 are the original algorithmic contributions. Read these files if asked "show me the novel part".

6.1 Clause decomposition - domain_inference.py

The problem: people write briefs as one long sentence. Splitting on sentences is not enough.

Function	Handles	Example
<code>_split_where_clauses()</code>	"<anything> where A does P, B does Q, and C does R"	The single commonest brief shape
<code>_split_subject_verb_list()</code>	One subject, many actions	"Drivers discover stations, reserve slots, pay securely, cancel bookings and request refunds." -> 5 requirements, all owned by Driver
<code>_is_product_title()</code>	The opening sentence that names the product	"EV charging station booking platform for fast-growing metro cities in India." is a title, not a requirement
<code>_split_enumeration()</code>	"... with A, B, and C" capability lists	Keeps the behaviour, drops the framing noun

The interesting part is the discriminator, because it needs no dictionary. To split "reserve charging slots" (an action) from "cards" and "proof of delivery" (more objects), `_looks_like_predicate()` uses English structure:

```

if words[0] in _CAPABILITY_VERBS:      return True      # known verb
if len(words) < 2:                      return False
if words[0].endswith("s"):              return False    # a plural noun heads a noun phrase
if words[0] in _ENGLISH_STOPWORDS:     return False
return words[1] not in {"of", "and", "on", "for", "in", "on", "with"}

```

Why this matters: a verb list can never be complete. Real briefs use "reserve", "pay", "request", "publish", "enroll", "score", "triage", "disburse" - none of which any curated list here contains. This rule handles all of them, because *a base-form verb is not plural and is not followed by a noun-phrase marker*. That is a property of English, not of a vocabulary, so it generalises to domains the project has never seen.

6.2 Actor↔use-case association - diagram_generator.py

Linking a requirement to the actor who performs it is the traceability backbone. Three parts:

A purpose-built stemmer (`\_stem`). A general stemmer is too aggressive; no stemming at all misses everything. The requirement was: *two spellings of the same idea must agree, and two different ideas must not*.

- ordered suffix list, longest first (so reservations -> reserv, **not** reservation)
- two passes**, because a word can carry two suffixes (discovery -> discover -> discov)
- bounded post-normalisation: doubled consonant (cancellation -> cancell -> cancel), trailing e (reserve -> reserv), trailing at (operator -> operat -> oper)

- a small table of English irregulars no suffix rule recovers (submission/submit, verification/verify, prescription/prescribe)
- **every normalisation stops at four characters**

That last rule is the design insight worth saying out loud:

***Over-stemming is worse than under-stemming here.** A short stem collides with unrelated words and invents an association - and an invented association is a wrong diagram, whereas a missed one is only an incomplete diagram. So the algorithm is deliberately tuned to fail safe.*

Verified on 28 word pairs with zero collisions across 35 domain nouns.

Containment, not overlap (``_containment``). Scores *how much of the requirement* the actor's vocabulary explains, rather than symmetric overlap - because symmetric overlap punishes a short requirement matched against a fuller actor description, and measurably dropped real associations. **Two shared stems minimum:** one shared word is not evidence, since "charging" appears in nearly every requirement of a charging project.

Evidence precedence. The actor's own *name* in the requirement wins outright (`_names_actor`); responsibilities are weighted next; the free-text description is deliberately weighted at **0.5**, because an actor with a long description otherwise outscores everyone and collects every use case.

6.3 Requirement -> artifact causal graph - `causal_graph.py`

Every generated element becomes a node; every derivation becomes an edge. This gives, by **graph traversal and not by asking a model**:

- traceability - why does this component/table/endpoint exist?
- blast radius - what breaks if this requirement changes?
- orphan detection - which components have no originating requirement? (surfaced as orphan-component consistency issues)
- counterfactual simulation - what if this requirement were removed?

`GRAPH_VERSION` is stored with the graph, so a stored graph built by an older version is recognised and rebuilt rather than misread.

6.4 UML correctness as a specification - `diagram_generator.py`

Most generators emit a flowchart and label it "use case diagram". Here the notation is treated as a spec, tested per diagram type across five domains (`tests/test_diagram_notation.py`, 140 tests):

- **Use case** - Mermaid has no use case diagram type at all, so the backend emits a structured `UseCaseModel` and `frontend/src/components/diagrams/UseCaseDiagram.tsx` draws real UML: stick figures outside a system boundary rectangle, ellipses inside, and undirected association paths. The frontend routes each association through a per-actor lane outside the boundary and then horizontally into the ellipse edge. This keeps connection paths away from actor labels and use-case text.
- **Activity** - start node -> **fork bar** -> per-actor **partitions** -> **join bar** -> final node. The fork/join is an honesty mechanism: the requirement model records no execution order, and a chain of arrows would invent one. UML's fork and join say *"these happen, order unspecified"* - which is exactly what the model supports.
- **Class** - `+name` : Type with UML types, never SQL types. A Mermaid member containing `()` is parsed as an *operation*, so `VARCHAR(255)` used to render every column in the methods compartment.
- **ER** - Mermaid's real grammar type `name [key] ["comment"]`, with PK/FK in the key slot. `*id` and `VARCHAR?` were invented notation that Mermaid does not parse.

- **Deployment** - «node» containing «artifact»s, with replicas and regions as **tagged values on the node**, joined by an undirected **communication path**. Previously the replica count was drawn as a box with an arrow into the deployment model.

Verification harness: all seven diagrams for all five domains are pushed through the application's own Mermaid build and both `mermaid.parse()` **and** `mermaid.render()` are required to succeed - **35/35**. Parsing alone is not enough; a diagram can parse and still fail to render.

6.5 Honest uncertainty as a first-class output

Every artifact distinguishes confirmed from unconfirmed, and says so in the artifact itself rather than in a footnote:

- an unassociated use case is drawn with a **dashed** ellipse, and the legend counts them
- an activity partition with no identified actor is titled "**Actor not confirmed**" - never the extractor's internal placeholder
- omitted requirements are **reported as a count**, not silently dropped
- `analysis_source` and `analysis_warnings` travel with the model, so the UI can show how the requirements were obtained
- `consistency_issues` surfaces contradictions instead of resolving them silently; when the database generator has to pick a relationship direction, it emits a note saying a direction was chosen for the reader

`tests/test_diagram_notation.py::test_no_diagram_prints_an_extractor_placeholder` asserts across all five domains that no internal placeholder string (`needs clarification`, `unknown`, `tbd`, ...) ever reaches a diagram.

7. Likely viva questions, with answers

Q: Isn't this just a wrapper around Ollama? No. Turn Ollama off and everything still works - that is how the 494 backend tests run. The model contributes semantic interpretation of unusual prose; it decides nothing. Every number it emits must already appear in the brief or it is deleted (`_unsupported_numbers`).

Q: What happens if the model returns rubbish? Three layers. Pydantic rejects the wrong shape (`UnknownDomainExtraction.model_validate`), grounding deletes untraceable content, then the deterministic classifier re-decides every label anyway. If all of that leaves nothing, the conservative fallback keeps the user's own words and raises clarification questions.

Q: How do you separate functional from non-functional requirements? Three signals - quality evidence (counted, not boolean), a hard-constraint test that is an AND of a modal and a solution-space marker, and a domain-action count - combined in a fixed order, with a dedicated disambiguator for words like "availability" that mean different things in different sentences. It re-decides every statement from its text rather than trusting the label it arrived with. 44/44 on the committed benchmark.

Q: How accurate is it, really? On the committed 44-statement benchmark, 44/44. That benchmark is ours and it is small, so the number is a regression guard rather than a general accuracy claim. What I can defend without qualification is that it is **deterministic**: the same statement always classifies the same way, the decision is inspectable line by line, and a regression fails a test. Before the fixes in Part III the same rules scored 36/43 - the benchmark is what caught that.

Q: Does it only work for your four domains? No. The four are *classifier labels*, not content. `_analyze_deterministic` handles anything, and the notation tests prove it on an LMS, freight logistics and an IT service desk - none of which has a blueprint. A blueprint contributes only a domain name.

Q: Where is the novelty, in one file? `domain_inference.py` for clause decomposition (splitting a brief into per-actor requirements with no verb dictionary), and `diagram_generator.py` for the actor-association stemmer and UML-correct construction. If only one: `domain_inference.py`.

Q: Why not let the LLM build the diagrams? Because notation is a specification. A model produces something diagram-shaped that is subtly wrong - a flowchart labelled "use case", SQL types in a class diagram - and you cannot unit-test it. The deterministic generator is checked by 130 notation tests and a render harness.

Q: What is the biggest weakness? Entity extraction. It still emits verbs as entities (`submit`, `enroll`), near-duplicates (`report` alongside `progress_report`) and non-entities (`discovery`, `city`). Passive-voice briefs are not decomposed. A brief that is only a product title falls back to echoing the brief with clarification questions. These are known, listed, and none of them is hidden from the user - they surface as consistency issues.

Q: How do you know a change did not break traceability? The causal graph is rebuilt and validated; orphan nodes (components with no originating requirement) become orphan-component consistency issues. Use case requirement ids are asserted to point at requirements that exist, across all five test domains.

Q: Why is the requirement model not regenerated when you improve extraction? Because it is the source of truth and the user edits it - silently re-extracting would delete their work. Derived artifacts (diagrams) do self-heal on read. For the requirement model there is an explicit, previewed, undoable `repair_requirement_model` action that removes only what can be *proven* to be an extraction artifact. See Part III.

Part III - Bugs Found and Fixed (with evidence)

Each entry is a real defect, reproduced before it was changed, with the file, the cause, the fix, and the test that now pins it. Nothing here is a speculative improvement.

How they were found

Three harnesses, because the existing tests all exercised one happy-path project:

5. **A labelled classification benchmark** - 44 statements tagged by what a requirements engineer would call them, fed in deliberately mislabelled.
6. **An adversarial brief sweep** - 16 deliberately awkward briefs (single sentence, no actor, all-quality, repeated title, semicolons, unicode, passive voice, ALL CAPS, a question, auth acronyms) checked against cross-artifact invariants rather than expected text.
7. **A Mermaid render harness** - every diagram for every domain pushed through the app's own Mermaid build, requiring both `parse()` and `render()` to succeed.

Bug 1 - A mandated technology stayed a functional requirement

File: `app/services/requirement_analyzer.py, _reclassify_requirements()`

Symptom: "Must use PostgreSQL as the primary datastore." was filed as a *functional requirement*. So was "Must deploy to Azure only.", "Cannot use any managed cloud queue service.", "The build shall use Java 17." Constraint recall on the benchmark was **2 / 8**.

Cause: one condition in the functional branch.

```
if quality_hits and _solution_constraint(item): # <- the bug
    moved_constraints.append(item)
```

The constraint test was only *consulted* when the statement also contained a quality word. "Must use PostgreSQL" contains no quality word at all, so `quality_hits == 0` and the branch was never evaluated - the statement fell through to `kept_functional` and stayed there forever.

Fix: evaluate the constraint test on its own merits.

```
if _solution_constraint(item):
    moved_constraints.append(item)
```

Why widening it is safe: `_solution_constraint` is already an AND of two independent signals - a hard modal **and** a solution-space marker. A domain action with a modal ("Operators must approve refunds.") matches no solution marker, so it stays functional. That is asserted directly.

Impact: constraint recall 2/8 -> **8/8**; overall 36/43 -> **44/44** on the benchmark as committed.

Tests: `test_a_mandated_technology_labelled_functional_becomes_a_constraint`,
`test_a_domain_action_with_a_modal_is_not_pulled_into_constraints`

Bug 2 - "must run nightly" was read as a deployment constraint

File: same function

Symptom: "Backups must run nightly with verified restores." - a recoverability *quality* - was filed as a solution constraint.

Cause: `must run` is in `SOLUTION_CONSTRAINT_MARKERS` because "must run on Kubernetes" is a genuine deployment constraint. The marker is matched as a **substring**, so "must run nightly" matched it too.

Fix: a placement marker followed only by a cadence is about *when* work happens, not *where* it runs.

```
placement = {"must run", "must host", "must deploy",
             "only run", "only host", "only deploy"}
if set(matched) <= placement and any(adverb in lower for adverb in schedule_adverbs):
    return False # nightly, hourly, weekly, overnight, periodically, ...
```

set(matched) <= placement matters: if the statement *also* matched a real marker such as `must` use or compliance, it is still a constraint.

Tests: `test_a_cadence_is_not_a_deployment_constraint` - including "The service must run on Kubernetes only." still classifying as a constraint.

Bug 3 - A brief with one entity had everything discarded

File: `app/services/requirement_analyzer.py, _analyze_deterministic()`

Symptom: "A tool where users submit forms and reviewers approve them for a small office team." produced **one** requirement - the brief verbatim - and **zero** actors.

Cause: the gate into the conservative fallback was an **OR**:

```
if len(entity_ids) < 2 or len(capabilities) < 1:
    return None # -> conservative fallback
```

That brief yields one entity (form), but **two clean capabilities and two actors**. A single entity threw all of it away, and the conservative fallback's entire output is the brief verbatim as FR-001 with no actors - strictly worse than the model it replaced. This is what made a one-entity brief look unanalysed.

Fix: fall back only when there is genuinely nothing to model.

```
if len(capabilities) < 1:
    return None
if len(entity_ids) < 1 and len(grounded_actors) < 1:
    return None
```

Result: 1 requirement / 0 actors -> **2 requirements / Reviewer extracted**.

Test: `test_one_entity_with_actors_is_not_discarded`

Bug 4 - A 500 when a brief names no business object

File: same function, domain-label derivation

Symptom: `POST /api/v1/workspaces` returned **500** for "Operators review equipment service records."

Cause: introduced by Bug 3's fix. `entity_ids` can now legitimately be empty, and the domain-label fallback indexed it unconditionally:

```
else f"{to_display_name(entity_ids[0])} Management Platform" # IndexError
```

Fix: three explicit cases - title, then most frequent concept, then "Unknown domain" (the label the rest of the pipeline already treats as unclassified) - and the warning text no longer interpolates a concept that may not exist.

Test: `test_a_brief_naming_no_business_object_does_not_crash`

Worth saying in a viva: this bug was *caused by* the previous fix and *caught by* the adversarial sweep within one run. It is the argument for having the sweep at all.

Bug 5 - "Unknown domain" printed as the system's name in three diagrams

Files: app/services/architecture_generator.py, app/services/diagram_generator.py

Symptom: for an unclassified brief the diagrams read:

```
use_case:    subgraph SYS["Unknown domain"]
sequence:    participant System as Unknown domain Core
component:    UNKNOWN_DOMAIN_CORE["Unknown domain<br/>Core Domain services"]
```

An internal placeholder was being presented as the name of the user's system.

Cause: RequirementModel.domain carries the literal string "Unknown domain", and nine f-strings in architecture_generator.py interpolated it into component names and overviews, from where it flowed into the sequence and component diagrams.

Why not just change the field: domain is load-bearing. project_signals.py branches on {"unknown", "unknown domain"} and two tests assert the literal. Changing it would break the signal layer.

Fix: a display helper at each boundary, leaving the field untouched.

```
def domain_label(domain: str) -> str:
    """A display name for the domain that never asserts an unknown one."""
    label = " ".join(str(domain or "").split())
    return "System" if label.casefold() in _UNCLASSIFIED_DOMAIN_LABELS else label
```

Applied to all nine interpolations, plus _system_label() in the diagram generator for the use case boundary and the sequence description.

Test: test_an_unclassified_domain_is_never_printed_in_a_diagram

Bug 6 - A brief that repeated its own title smuggled it back in

Files: domain_inference.py, requirement_analyzer.py

Symptom: "Fleet telemetry platform. Fleet telemetry platform." - the title was correctly skipped as sentence 0 and then accepted as sentence 1.

Cause: the title was skipped **by position**.

Fix: match it **by text**, so a repeat is skipped wherever it appears.

```
title_key = ""
if len(sentences) > 1 and _is_product_title(sentences[0]):
    title_key = " ".join(sentences[0].casefold().split()).rstrip(".")
for sentence in sentences:
    if title_key and " ".join(sentence.casefold().split()).rstrip(".") == title_key:
        continue
```

Test: test_a_repeated_title_sentence_is_skipped_everywhere

Bug 7 - A brief not phrased as an instruction was never decomposed

File: domain_inference.py, _WHERE_CLAUSE_LEAD / _split_where_clauses()

Symptom: "A tool where users submit forms and reviewers approve them." stayed one requirement. "Build a tool where ..." split correctly.

Cause: the lead-in pattern required an imperative opener (build|create|develop|design|implement). A brief that names the product as a noun phrase - extremely common - never matched.

Fix (two parts):

8. Allow a determiner-led noun phrase as the lead-in: `(?:an?|the)\s+[\^, .]{0,60}? \b(?:where|in which)\s+`
9. With no commas, try and - but only when **both** sides are independent clauses with their own plural subject:

```
def _has_plural_subject(segment: str) -> bool:
    words = tokenize(segment)
    if len(words) < 3:
        return False
    subject = words[0]
    if not (subject.endswith("s") and not subject.endswith("ss")):
        return False
    return _looks_like_predicate(" ".join(words[1:]))
```

That guard is why "station and charger availability" does **not** split ("station" is singular and one token) while "users submit forms and reviewers approve them" does.

Tests: `test_a_brief_that_opens_with_a_noun_phrase_still_splits`,
`test_and_splits_two_clauses_but_never_one_object`, `test_plural_subject_detection` (6 cases)

Bug 8 - Widening the title rule discarded a whole capability list

File: `domain_inference.py`, `_is_product_title()`

Symptom: "Build a cold-chain slot coordination tool with depot discovery, live bay availability, slot booking, and cancellation requests." produced **zero** functional requirements.

Cause: the title rule now recognises a product head noun anywhere in the sentence; "tool" matched, no role noun was present, and no finite verb was found - so the sentence was classified as a title and its entire capability list went with it. The imperative guard should have prevented that, but it checked only `_CAPABILITY_VERBS`, which does not contain **"build"** - the single commonest way a brief begins.

Fix: an explicit set of imperative openers.

```
_IMPERATIVE_OPENERS = frozenset("""
    build create develop design implement make deliver produce launch ship
    add allow enable ensure expose extend generate integrate introduce
    let offer provide rebuild refactor replace set setup support
    """).split()
```

Test: `test_an_imperative_brief_is_not_mistaken_for_a_title`

Worth noting: Bugs 4 and 8 were both *caused by* fixing an earlier bug and *caught by the harness in the same session*. That is the case for measuring rather than assuming.

Bug 9 - The test database was never reset

File: `backend/tests/conftest.py`

Symptom: the suite slowed down run over run and was eventually **killed for memory** - which presents exactly like a hanging test.

Cause: nothing deleted `test_archai.db`, so it accumulated every workspace any run had ever created: **2,933 workspaces, 1.7 GB**. The list endpoint loads and repairs each workspace it returns, so the cost grew with every run.

Fix: delete the file (and its `-wal` / `-shm` sidecars) before the run. Two consecutive runs now hold steady at ~115 s.

Bug 10 - Actor cleanup and multi-actor associations were lost

File: `app/services/diagram_generator.py`

Symptom: a user rename such as Driver to USER rendered as all-caps, authentication labels such as Sso Admin appeared as extra actors, and a use case explicitly involving both operators and administrators was connected to only one of them.

Cause: the use-case generator had fallen back to the first matching actor and bypassed the existing profile normalization and alias evidence.

Fix: restore `_use_case_actor_profiles()`, `_actors_for_use_case()`, and `_actor_aliases()`. Actor profiles now normalize display names, remove access mechanisms from roles, merge duplicates, preserve user-edit rename evidence, and return every explicitly confirmed participant. Actors with no association are not drawn as decorative figures.

Tests: `test_actor_renames_duplicates_and_shorthand_roles_produce_a_clean_model` and `test_one_use_case_can_have_multiple_confirmed_participants`.

Bug 11 - PlantUML activity exports invented a sequence

File: `app/services/diagram_generator.py`, `_dynamic_activity()`

Symptom: the Mermaid activity view used a fork and join when several workflows had no confirmed order, but the PlantUML export listed those same workflows one after another. The two exports contradicted each other, and the PlantUML version asserted causality the requirements never supplied.

Fix: each workflow is now a PlantUML fork or fork again branch inside its actor partition, closed by end fork. A single workflow remains sequential.

Test: `test_activity_plantuml_export_does_not_invent_workflow_order`, run across EV charging, pharmacy, learning, logistics, and helpdesk briefs.

Bug 12 - Component diagrams contained a fake runtime component

File: `app/services/diagram_generator.py`, `_component()`

Symptom: the Mermaid component diagram drew an Architecture box with dotted contains arrows to every component. That box is a presentation label, not a deployable runtime component, and duplicated the tier subgraphs already expressing containment.

Fix: remove the synthetic hub and its edges. The diagram now contains only generated runtime components and their confirmed dependencies, grouped into tiers. If none are confirmed, it states that explicitly instead of drawing a fake component.

Test: `test_component_diagram_contains_only_runtime_components`, also run across the five unrelated domains.

Verification after all fixes

Check	Result
Backend tests	494 passed
Frontend tests	39 passed across the main run and isolated Windows worker

Check	Result
	retry
TypeScript (tsc -b)	clean
Production build	succeeds
Classification benchmark	44/44 (was 36/43)
Diagram parse and render	35/35 across 5 domains
Adversarial brief sweep	13/16 clean; the other 3 are correct behaviour on degenerate input (see below)
Route sweep	16/16 routes, 0 console errors

The three sweep flags that are not bugs

Stated explicitly, because they look like failures in the harness output and an examiner may ask:

10. ``all-nfr`` - a brief stating only qualities yields **0 functional requirements**. Correct: there is no behaviour in the brief. (*Known limitation: it produces one NFR rather than four; the enumeration splitter does not yet run over quality lists.*)
11. ``long-noun-phrase`` - a brief that is a pure noun phrase of quality adjectives yields 0 functional requirements. Correct, same reason.
12. ``title-only-multi`` - a brief that is *only* a repeated product title falls to the conservative fallback, which keeps the brief verbatim and raises clarification questions. That is the fallback's documented job: there is no requirement in the input to extract.

Known limitations, stated deliberately

- **Entity extraction** emits verbs as entities (submit, enroll), near-duplicates (report alongside progress_report) and non-entities (discovery, city).
- **Passive-voice briefs** are not decomposed. "Applications are reviewed by underwriters and decisions are communicated to brokers." stays one requirement.
- **Quality enumerations** are not split into separate NFRs.
- **Table naming** is inconsistently plural/singular within one schema (BOOKINGS, CHARGERS beside CITY, SLOT).
- ``USE_CASE_LIMIT`` is **12**; beyond that, the remainder is reported as a count rather than drawn.
- Domain classification can pick a secondary capability over the primary one (a hotel brief classified as "Housekeeping Management Platform").

Part IV - Demonstration script and cheat sheet

A 4-minute live demo that proves the claims

Run this order; each step answers a question before it is asked.

1. Prove it is not an LLM wrapper (45 s). Stop Ollama. Open Settings - the health check shows `ollama_reachable: false`. Create a project from a domain that has no blueprint:

"Build a grant management system where reviewers score applications, committees approve awards, and finance teams disburse funds, track milestones and reconcile receipts."

Point at the Requirements page badge: **"Extracted deterministically from the brief wording"**. Four requirements, one per actor clause, each attributed. Seven diagrams. No model was involved.

2. Prove it does not invent facts (30 s). Same project. Point out there is no availability target, no traffic number, no named cloud - because the brief contains none. Then say: *every number in the output must appear in the input; it is a set difference in `\_unsupported\_numbers`, four lines.*

3. Prove FR/NFR separation is a decision, not a keyword (45 s). Add two requirements by hand:

- "Display live bay availability for each depot." -> stays **functional**
- "The service must meet a 99.95% availability target." -> becomes **non-functional**

Same word, "availability", opposite classifications. Then name the disambiguator: `_resource_availability_capability`.

4. Prove traceability (45 s). Open the Causal graph - every artifact traces to its originating requirement. Open Blast radius and change one requirement: the affected set is computed by graph traversal, not by a model.

5. Prove the diagrams are UML, not decoration (45 s). Open Diagrams -> Use Case: stick figures outside a system boundary, ellipses inside, association lines meeting the outline exactly. Switch to Activity: the fork and join bars. Say: *the fork is there because the requirement model records no execution order - a chain of arrows would invent one.*

6. Prove it is tested (30 s). `npm test` -> 494 backend, 39 frontend. Mention the 44-statement classification benchmark and the 35/35 Mermaid render harness.

Cheat sheet - question -> file -> the one line to say

If asked about	Open this	Say this
Novelty	<code>services/domain_inference.py</code>	"Clause decomposition with no verb dictionary - it uses English structure, so it generalises to domains we have never seen."
FR vs NFR	<code>requirement_analyzer.py</code> -> <code>_reclassify_requirements</code>	"Three signals in a fixed order; it re-decides every statement from its text instead of trusting the label it arrived with."
Accuracy	<code>tests/test_requirement_classification.py</code>	"44/44 on a committed benchmark - small, ours, and a regression guard"

If asked about	Open this	Say this
		rather than a general claim. It is what caught the 36/43 before the fixes."
Preventing hallucination	requirement_analyzer.py -> _strip_unsupported_claims	"Five independent checks; any one fires and the text is discarded. Numbers are a set difference against the brief."
Static or dynamic	requirement_analyzer.py -> analyze / _pick_blueprint	"Dynamic. Four blueprints exist, but a blueprint is a classifier, never a content source - and the code comment says exactly that."
Why deterministic	tests/conftest.py	"The whole suite runs with Ollama disabled. If the intelligence were in the prompt, none of it would pass."
Traceability	services/causal_graph.py	"Every artifact is a node, every derivation an edge; blast radius is graph reachability, not a model's guess."
Diagram correctness	services/diagram_generator.py + tests/test_diagram_notation.py	"Notation is treated as a specification - 140 tests across five domains, plus a harness that requires Mermaid to both parse and render."
Use case diagram	frontend/.../UseCaseDiagram.tsx	"Mermaid has no use case diagram type, so the backend emits a structured model and this draws real UML with orthogonal association lanes that avoid labels and enter at the ellipse edge."
Handling uncertainty	diagram_generator.py -> _UNKNOWN_ACTOR_LABELS	"A dashed ellipse and 'Actor not confirmed' - the diagram states what the model knows and nothing more."
Upgrading an old project	services/requirement_repair.py	"The requirement model is the source of truth and the user edits it, so it is never silently re-extracted. There is an explicit, previewed, undoable repair that removes only provable artifacts."
Biggest weakness	-	"Entity extraction - it still emits verbs and near-duplicates as entities. It is listed as a known limitation and surfaces as a consistency issue, not hidden."

Numbers to have memorised

Metric	Value
Backend tests	494
Frontend tests	39
Classification benchmark	44/44 (was 36/43 before the fixes)
Diagrams parsed and rendered	35/35 (7 types x 5 domains)
Notation tests	140, across 5 unrelated domains
Routes with zero console errors	16/16
Deterministic backend service code	~12,900 lines across 9 services
Diagram types generated	7, in both Mermaid and PlantUML
Architecture metrics scored	12, weighted by scale profile
Domains with a blueprint	4 - labels only, no content
Domains the system can handle	unbounded (deterministic path)

Three sentences that answer most follow-ups

13. *"The language model is not allowed to decide anything - it interprets prose, and everything it returns is validated and grounded against the brief before it is stored."*
14. *"Every artifact traces back to a sentence the user actually wrote, and where the requirement model does not know something, the artifact says so instead of inventing a plausible answer."*
15. *"Because it is deterministic, it can be measured - which is how I know the classifier scores 44/44 and how the regression suites found and pinned the twelve documented bugs."*